# Programming Assignment 2

Data Prefetching and Cache Replacement in ChampSim

**Team VS CoPiLot**

Advay Bapat 23B1225
Aniket Gupta 23B1245
Apoorv Goyal 23B1266
Kundan Maiti 23B1242
Department of Electrical Engineering
Indian Institute of Technology Bombay

**CS683: Advanced Computer Architecture**

**Date:** October 15, 2026

# Contents

**1 Task 3: SPP with Prefetch-Aware Dynamic I-RRIP** **2**
1.1 What the L2 sees from SPP . . . . . 2
1.2 Design: fill-level-aware prefetch handling . . . . . 2
1.3 Rationale for Handling Prefetched Lines . . . . . 3
1.4 Results . . . . . 4
1.5 Discussion . . . . . 4

# 1 Task 3: SPP with Prefetch-Aware Dynamic I-RRIP

Task 3 runs the final Task 1 prefetcher (SPP, `prefetcher/spp.l1d_pref`, unchanged) at the L1D together with the dynamic I-RRIP policy of Task 2B at the L2, and makes that policy aware of the lines SPP brings in. All prefetch handling lives in `replacement/dyn_irrip.l2c_repl` behind one switch, `PF_AWARE`. With no prefetcher none of the new code runs, so the Task 2B results are unchanged.

## 1.1 What the L2 sees from SPP

Every SPP prefetch is issued at the L1D and, when it misses there, travels to the L2 as a `PREFETCH` request. SPP chooses its fill level from the L1D MSHR occupancy (Task 1):

- **FILL_L1** (L1D MSHR less than half full): the line is filled into the L2 *and* the L1D. The demand that follows hits in the L1D, so the L2 copy is a duplicate that the L2 will rarely be asked for again.
- **FILL_L2** (L1D MSHR at least half full): the line is filled into the L2 *only*. It is not in the L1D, so its demand access has to come to the L2.

The two kinds of prefetch therefore have very different value *to the L2*, even though both arrive with the same packet type. We measured this directly (Table 1, Figure 1): only 0.15–0.72% of FILL_L1 prefetch fills ever get a demand hit in the L2, while 13–19% of FILL_L2 prefetch fills do. FILL_L2 prefetches are only 1.5–4% of the prefetch fills, yet they account for about half or more of all L2 demand hits on prefetched lines.

Table 1: Prefetch fills into the L2 and their first demand use, split by fill level (SPP + prefetch-aware Dyn. I-RRIP, whole run including warm-up).

| | FILL_L1 (L1D + L2) | | | FILL_L2 (L2 only) | | |
|---|---|---|---|---|---|---|
| **Trace** | **Fills** | **First use** | **Used (%)** | **Fills** | **First use** | **Used (%)** |
| Trace 1 | 143403 | 1028 | 0.72 | 5455 | 1062 | 19.47 |
| Trace 2 | 94376 | 232 | 0.25 | 1421 | 230 | 16.19 |
| Trace 3 | 106602 | 158 | 0.15 | 3220 | 423 | 13.14 |

## 1.2 Design: fill-level-aware prefetch handling

The L2 fill path in `src/cache.cc` stores the fill level of the MSHR entry in a new `CACHE` member, `repl_fill_level`, just before it calls `update_replacement_state()` (one line in `cache.cc`, one in `inc/cache.h`). The replacement interface is otherwise unchanged. The policy then applies these rules in both follower modes (I-RRIP and RRIP) and in the leader sets:

1. **FILL_L1 prefetch fill** → RRPV 7 (`PF_L1_INSERT`), the most distant position. The line is a duplicate of an L1D copy and is the first candidate for eviction, so it cannot push out instruction lines or demand data.
2. **FILL_L2 prefetch fill** → inserted like a demand data miss of the current mode: RRPV 4 in I-RRIP mode, RRPV 6 in RRIP mode. The line has to survive in the L2 until its demand arrives.
3. **Prefetch hit in the L2** → RRPV unchanged. The request only copies the line up to the L1D, which says nothing about future L2 reuse (the Task 2B policy decremented the RRPV here).

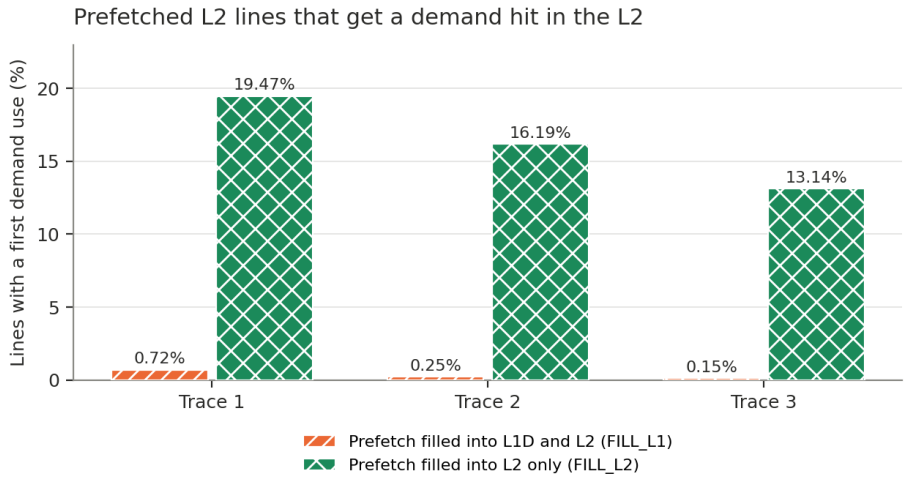



Figure 1: Share of prefetched L2 lines that receive a demand hit in the L2, by fill level. L2-only prefetches are 25–90× more likely to be used in the L2.

4. **First demand hit on a prefetched line** → promoted by the normal LOAD-hit rule of the mode (to RRPV $\leq 2$ for data in I-RRIP mode, to RRPV 4 in RRIP mode). The prefetch has proved useful, so the line is treated like any reused line.
5. **Set dueling sees demand traffic only.** `PREFETCH` requests are not counted in the leader-set access/miss counters or in the epoch length. Prefetch traffic depends on the prefetcher, not on how well the policy keeps instruction or data lines, so counting it would dilute the instruction ratio that the duel uses to choose the mode.

No per-line state is added for the policy itself: the RRPV array and the existing `BLOCK::prefetch` bit are enough (a 2-bit per-line tag is kept only for the statistics of Table 1). Table 2 summarises how each class of line is treated.

Table 2: Treatment of each line class (RRPV range 0–7) in the prefetch-aware policy.

| **Line class** | **Insertion (I-RRIP / RRIP mode)** | **Promotion on hit** | **Eviction priority** |
|---|---|---|---|
| Instruction (demand) | 1 / 6 | to 0 / to 4 | last: among RRPV-7 lines, data is evicted first in I-RRIP mode |
| Data (demand) | 4 / 6 | to $\leq 2$ / to 4 | normal |
| Prefetch, FILL_L1 | 7 / 7 | prefetch hit: none; first demand hit: as data | first (inserted at the eviction position) |
| Prefetch, FILL_L2 | 4 / 6 | prefetch hit: none; first demand hit: as data | as demand data |

## 1.3 Rationale for Handling Prefetched Lines

- **Most prefetched L2 lines are dead on arrival.** SPP is an L1D prefetcher: a useful prefetch is consumed in the L1D, and the L2 copy is not asked for again unless the L1D evicts it first. Under 1% of FILL_L1 prefetch fills are ever used in the L2 (Table 1).

Inserting them at the eviction position means that inaccurate or duplicate prefetches never displace the instruction lines that I-RRIP is protecting.

- **The useful L2 prefetches must not be thrown away.** A FILL_L2 prefetch is the only copy above the LLC. Inserting it at the distant position would evict it before its demand arrives and waste the DRAM access. Giving it the same position as demand data keeps it as long as any other data line. The 25–90× difference in L2 use between the two kinds of prefetch is the measured reason for treating them differently instead of treating "prefetch" as one class.
- **A prefetch hit is not reuse.** A prefetch request that hits in the L2 is about to copy the line into the L1D; promoting the L2 copy would keep a line that the L1D now serves. Only a demand hit is evidence of L2 reuse.
- **The duel must judge the policy, not the prefetcher.** Prefetch misses are data misses that both leader groups would suffer whatever the mode. Keeping them out of the counters makes the I-RRIP/RRIP decision depend only on demand behaviour, the same signal as in Task 2B.

## 1.4 Results

We compare four configurations, all built from the same code base and run with 25M warm-up and 25M simulated instructions: the baseline (no prefetcher, LRU at the L2), SPP with LRU (Task 1 final), dynamic I-RRIP without a prefetcher (Task 2B), and SPP with the prefetch-aware dynamic I-RRIP (Task 3, written SPP + Dyn. I-RRIP-PF in the tables). Speedup is over the baseline. Accuracy and coverage are defined as in Task 1 (coverage = baseline L1D load misses removed). Means are geometric for IPC and speedup and arithmetic otherwise.

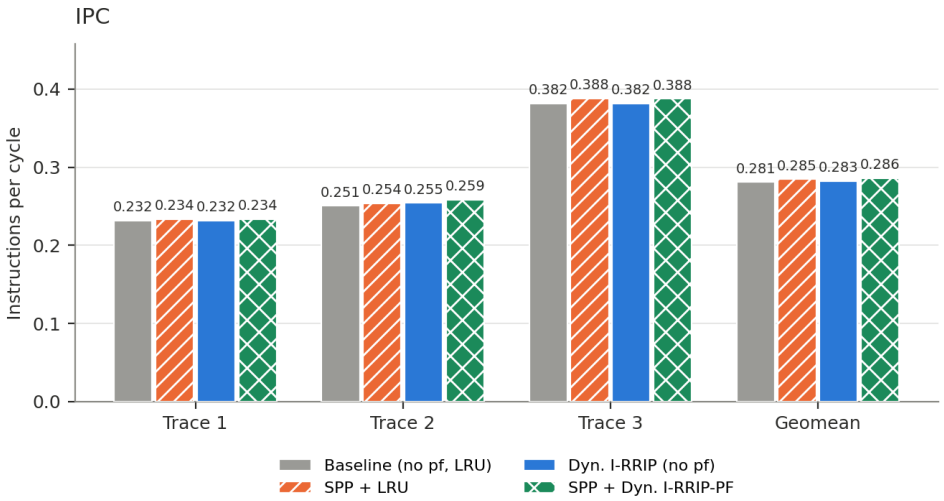



Figure 2: Task 3: IPC of each configuration.

## 1.5 Discussion

**Outcome.** SPP with the prefetch-aware dynamic I-RRIP reaches a geometric-mean speedup of **1.0186** (+1.86%), against 1.0129 for SPP alone and 1.0053 for dynamic I-RRIP alone, so it outperforms both on average. It beats dynamic I-RRIP on every trace.

Table 3: Task 3: IPC, speedup and L1D prefetch metrics.

| Trace | Configuration | IPC | Speedup | Accuracy (%) | Coverage (%) |
|---|---|---|---|---|---|
| Trace 1 | Baseline (no pf, LRU) | 0.2320 | 1.0000 | – | – |
| | SPP + LRU | 0.2339 | 1.0080 | 47.6 | 10.87 |
| | Dyn. I-RRIP (no pf) | 0.2320 | 1.0000 | – | – |
| | SPP + Dyn. I-RRIP-PF | 0.2338 | 1.0077 | 47.4 | 10.90 |
| Trace 2 | Baseline (no pf, LRU) | 0.2506 | 1.0000 | – | – |
| | SPP + LRU | 0.2541 | 1.0141 | 51.6 | 4.64 |
| | Dyn. I-RRIP (no pf) | 0.2547 | 1.0165 | – | – |
| | SPP + Dyn. I-RRIP-PF | **0.2586** | **1.0322** | 52.0 | 4.61 |
| Trace 3 | Baseline (no pf, LRU) | 0.3819 | 1.0000 | – | – |
| | SPP + LRU | 0.3882 | 1.0165 | 33.4 | 15.28 |
| | Dyn. I-RRIP (no pf) | 0.3817 | 0.9996 | – | – |
| | SPP + Dyn. I-RRIP-PF | 0.3880 | 1.0161 | 33.4 | 15.37 |
| Mean | Baseline (no pf, LRU) | 0.2811 | 1.0000 | – | – |
| | SPP + LRU | 0.2847 | 1.0129 | 44.2 | 10.26 |
| | Dyn. I-RRIP (no pf) | 0.2826 | 1.0053 | – | – |
| | SPP + Dyn. I-RRIP-PF | **0.2863** | **1.0186** | 44.3 | 10.29 |

Table 4: Task 3: load MPKI of the L1D, the L2C, and the L2 split into instruction and data loads.

| Trace | Configuration | L1D load | L2C load | L2 instr. | L2 data |
|---|---|---|---|---|---|
| Trace 1 | Baseline (no pf, LRU) | 10.68 | 17.72 | 10.37 | 7.35 |
| | SPP + LRU | 9.52 | 16.75 | 10.41 | 6.35 |
| | Dyn. I-RRIP (no pf) | 10.68 | 17.63 | 9.97 | 7.67 |
| | SPP + Dyn. I-RRIP-PF | 9.51 | 16.52 | 9.94 | 6.59 |
| Trace 2 | Baseline (no pf, LRU) | 13.54 | 28.19 | 19.58 | 8.61 |
| | SPP + LRU | 12.91 | 27.70 | 19.66 | 8.04 |
| | Dyn. I-RRIP (no pf) | 13.54 | 24.16 | 15.10 | 9.05 |
| | SPP + Dyn. I-RRIP-PF | 12.91 | 23.40 | 14.95 | 8.44 |
| Trace 3 | Baseline (no pf, LRU) | 3.40 | 3.96 | 1.13 | 2.84 |
| | SPP + LRU | 2.88 | 3.34 | 1.16 | 2.18 |
| | Dyn. I-RRIP (no pf) | 3.40 | 3.98 | 1.10 | 2.87 |
| | SPP + Dyn. I-RRIP-PF | 2.88 | 3.32 | 1.11 | 2.20 |
| Mean | Baseline (no pf, LRU) | 9.21 | 16.63 | 10.36 | 6.26 |
| | SPP + LRU | 8.44 | 15.93 | 10.41 | 5.52 |
| | Dyn. I-RRIP (no pf) | 9.21 | 15.26 | 8.72 | 6.53 |
| | SPP + Dyn. I-RRIP-PF | **8.44** | **14.41** | **8.67** | 5.75 |

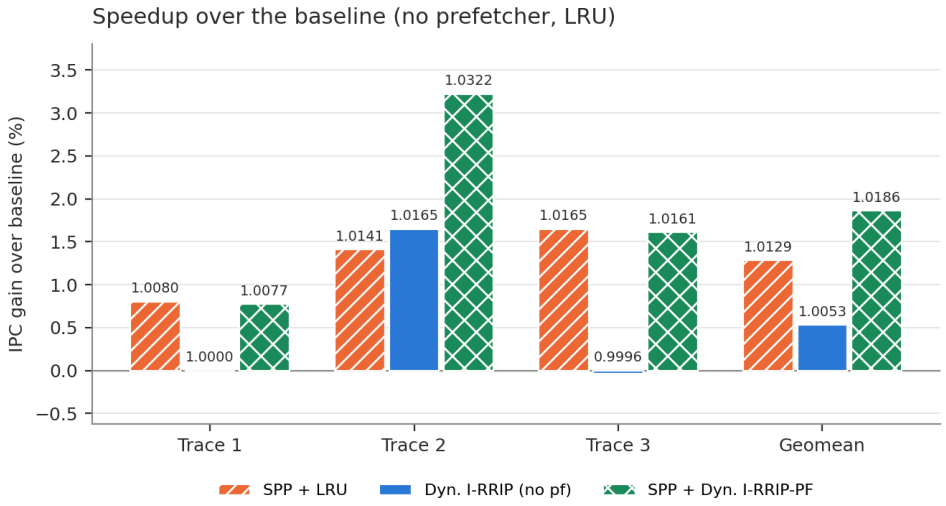



Figure 3: Task 3: speedup over the baseline (no prefetcher, LRU). Bar labels are the speedup.

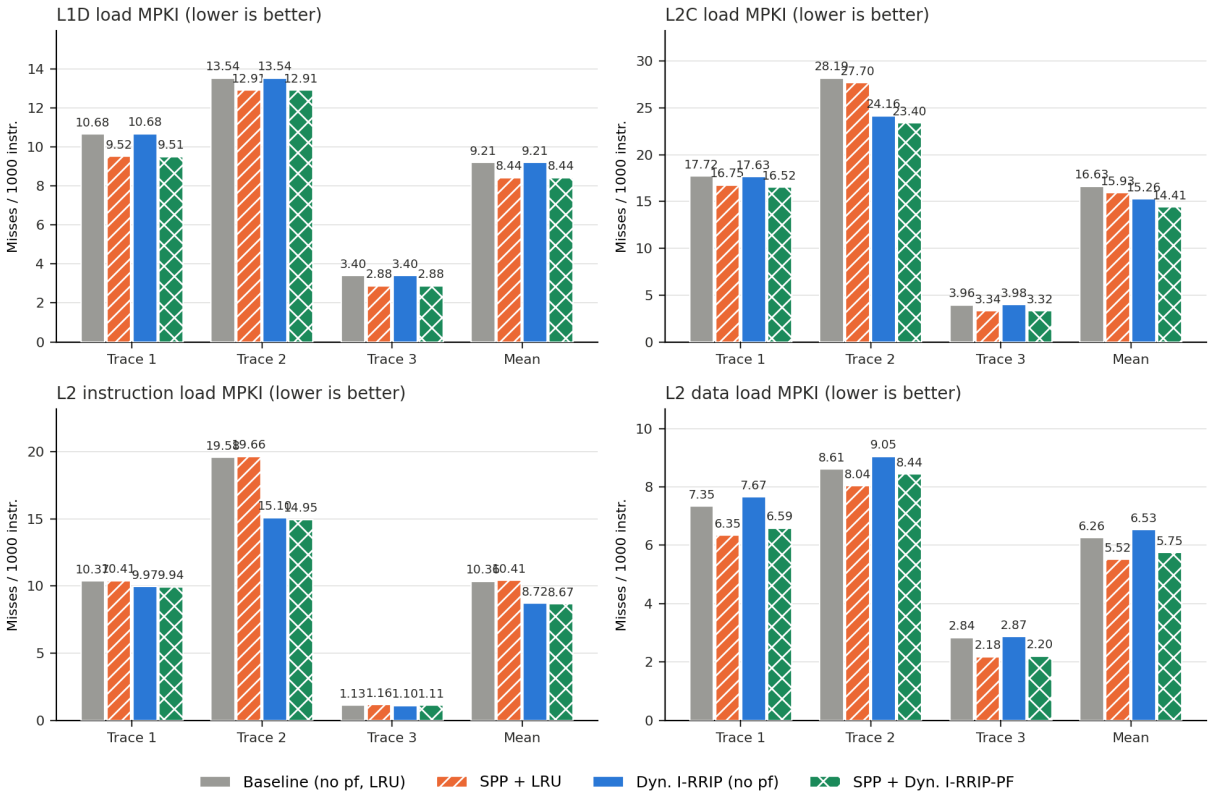



Figure 4: Task 3: load MPKI of the L1D, the L2C, L2 instruction and L2 data loads.

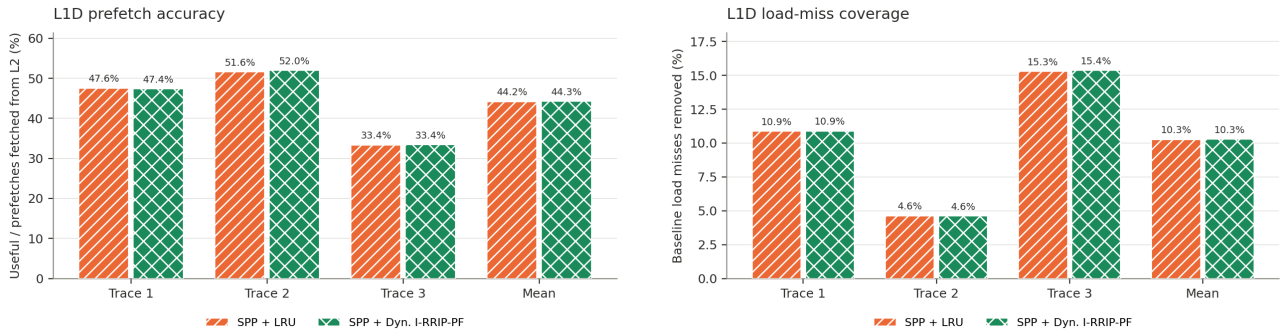



Figure 5: Task 3: L1D prefetch accuracy (left) and coverage (right) of SPP with LRU and with the prefetch-aware Dyn. I-RRIP at the L2.

**Why the gains add up.** The two mechanisms remove different misses. SPP removes L1D data misses (mean L1D load MPKI 9.21 → 8.44, coverage about 10%), which an L2 policy cannot touch. Dynamic I-RRIP removes L2 instruction misses (mean 10.36 → 8.67), which SPP cannot touch. On Trace 2, the most instruction-bound trace, SPP gives +1.41% and dynamic I-RRIP +1.65%; together they give +3.22%, slightly more than the product of the two (1.0308). The L2 instruction MPKI drops from 19.66 to 14.95 while the L1D load MPKI stays at the SPP level (12.91).

**SPP is not disturbed.** Accuracy and coverage are the same with LRU and with the new policy (mean accuracy 44.2% vs 44.3%, coverage 10.26% vs 10.29%). The L2 policy changes how long the L2 keeps lines, not what SPP predicts. Because FILL_L1 prefetches are inserted at the eviction position, the extra prefetch traffic does not cost I-RRIP its instruction-miss savings: the L2 instruction MPKI with SPP (8.67) is as low as without a prefetcher (8.72).

**Trace 1 and Trace 3.** On these traces dynamic I-RRIP alone gains nothing (1.0000 and 0.9996): Trace 3 has almost no L2 instruction misses (1.13 MPKI), and on Trace 1 the instruction misses saved (10.37 → 9.97) are paid back in data misses (7.35 → 7.67). With no instruction-miss saving to add, the combination can only match SPP, and it does so to within 0.04% (1.0077 vs 1.0080 and 1.0161 vs 1.0165, a difference of about 30k cycles in 107M on Trace 1). The set-dueling threshold of Task 2B still selects I-RRIP for most epochs on these traces (97% on Trace 1, 75% on Trace 3); a stricter mode decision is the natural next step for these workloads.

**Cost of distant insertion.** Inserting FILL_L1 prefetches at RRPV 7 lowers the L2 hit rate of *prefetch requests*: when SPP re-prefetches a line that the L1D has already evicted, the L2 copy has often gone too (Trace 1: prefetch-request hit rate 27% with LRU vs 18%). These re-prefetches go to the LLC instead. The effect on IPC is small, since the demand hit rate of the L2 rises at the same time (Trace 1 L2 load MPKI 16.75 → 16.52).